前置知識與目標

- 前置知識為 binary heap、鏈結樹、歸納證明、對數與攤銷分析。
- 能由 meld（合併）需求推導不同堆積結構，追蹤 leftist、binomial、Fibonacci 及標準 two-pass pairing heap。
- 能區分真正執行步數、最壞界與攤銷界，並明訂 decrease-key 是否已有節點 handle。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mergeable priority queue 的契約

- 堆積狀態是含重複鍵的多重集合。meld 合併兩堆所有元素，保留各元素身分。
- 本章以 destructive meld 為主，合併後來源堆不再擁有節點。若要求保留兩份可獨立修改的來源，成本不同。
- decrease-key 先取得合法節點 handle，再降低鍵值。搜尋任意鍵的成本不能藏在 `O(1)` 或 `O(log n)` 之內。
- 多個相同最小鍵時可回傳其中任一個，若要穩定順序，另以插入序號組成複合鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Leftist heap：刻意讓合併走短路

將兩個 binary heap 的陣列串接後重新建堆，要花線性時間。Leftist heap 改用鏈結二元樹：**不要求整棵樹矮，只要求反覆往右走，很快就遇到空指標。** meld 每層只走其中一棵樹的右子樹，這正是要縮短的路。

令空樹的 rank 為 0，非空節點滿足

$$r(x) = 1 + \min\{r(x.left), r(x.right)\}.$$

葉的 rank 為 1；只有左子的節點仍為 1。rank 衡量通往空子位置的最短路徑，**不是高度，也不是子節點數。**

Leftist heap 同時要求父鍵不大於子鍵，以及 $r(left) \geq r(right)$。因此 $r(x) = 1 + r(x.right)$，rank 恰好等於右鏈上的非空節點數。只向左延伸的 n 點鏈是合法形狀，rank=1，高度卻為 n−1。

Meld：先留下小根，回程再調整左右

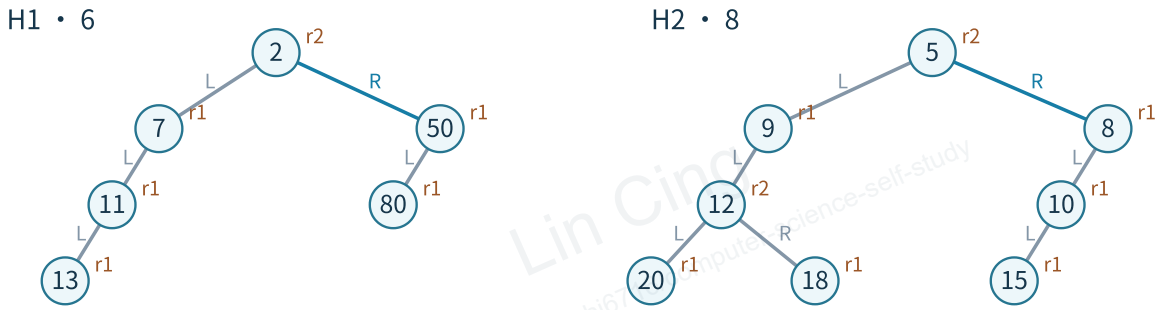
兩棵輸入須為合法、節點互不重疊的堆。下列採 destructive meld；相等鍵保留第一個根。節點記錄含 key、left、right、rank，不含 parent。

```
rank(null) = 0
meld(a, b):
    if a == null: return b
    if b == null: return a
    if a.key > b.key: swap(a, b)
    a.right = meld(a.right, b)
    if rank(a.left) < rank(a.right):
        swap(a.left, a.right)
    a.rank = 1 + rank(a.right)
    return a
```

下去時比較根；回來時比較 rank。 小根 a 的左子樹先保留，右子樹交給遞迴合併。回傳後才知道新右子樹的 rank，這時才決定是否交換子指標。swap 不交換 key 或 payload。

合併輸入：14 個節點，兩條待走的右鏈

節點內是 key，旁邊 r 是 rank；L、R 指明子指標。缺邊代表 null，rank=0。藍線標出原右鏈，其他邊不是本次遞迴向下走的路。



H1 有 6 點，右鏈 2→50；H2 有 8 點，右鏈 5→8，兩根 rank 都是 2。9 的左子樹可以很深，它的 rank 仍只有 1，因為右子指標為空。

遞迴向下：先不急著交換子樹

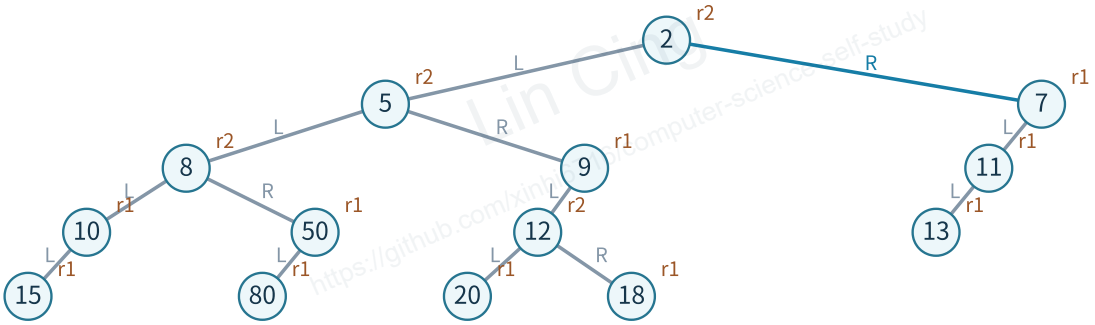
每次保留較小根，只把它的右子樹送入下一層。回傳前，尚未更新保留根的 rank。

呼叫	根比較與保留者	保留的左子樹	下一個呼叫
meld(2,5)	2<5，保留 2	根 7	meld(50,5)
meld(50,5)	5<50，交換局部根後保留 5	根 9	meld(8,50)
meld(8,50)	8<50，保留 8	根 10	meld(null,50)
meld(null,50)	空樹基例，不比較 key	—	回傳根 50

只做 **3 次根鍵比較**。2、5、8 的呼叫仍未結束；不能把暫時接上的 min-heap 當成已修復的 leftist heap。接下來依 8→5→2 的順序回傳。

回傳修復：為什麼要在 5 與 2 交換

回到的根	接回後的左、右根	左、右 rank	動作與新 rank
8	10、50	1、1	不交換，r(8)=2
5	9、8	1、2	交換成左 8、右 9，r(5)=2
2	7、5	1、2	交換成左 5、右 7，r(2)=2



為什麼 meld 的結果一定正確

對兩棵輸入的總節點數做歸納。基例有一棵為空，回傳另一棵，節點集合與兩項不變量均正確。皆非空時選較小根 a，遞迴只處理 a 的右子樹與另一堆，總節點數至少少一。

1. **元素不丟失、不重複。** 互不重疊的輸入分成 a、原左子樹與遞迴輸入。歸納假設保留遞迴中的每個節點；接回與交換子指標不增刪節點。
2. **Heap order 保持。** a 不大於原左右子樹內任何鍵，也不大於另一堆的根及其後代。因此 a 不大於新子根；子樹內依歸納假設合法。左右交換不改這些鍵比較。
3. **Leftist 性質恢復。** 兩棵子樹已合法，將 rank 較大者放左；設定 $r(a) = 1 + r(a.right)$ 就等於定義式。

總節點數嚴格減少也保證終止。meld(H,H) 違反節點互不重疊的前提，可能產生自環；必須在堆的所有權介面排除，不能把它當成複製堆。

最壞成本：只走兩棵輸入的原右鏈

先證 rank 界。若子樹 rank 至少 r，兩子樹 rank 都至少 r−1。令 N(r) 是 rank 至少 r 所需的最少節點數；N(0)=0，利用歸納假設得

$$N(r) \geq 1 + 2N(r-1) \geq 1 + 2(2^{r-1} - 1) = 2^r - 1.$$

所以含 n 點的堆滿足 $r \leq \lfloor \log_2(n+1) \rfloor$。基例 r=0 成立。

遞迴下降時還沒做回程交換。每一步只把其中一根換成它的**原右子樹**，兩個當前 rank 的和恰減 1。兩者皆非空時和至少為 2；輸入 rank 為 r1、r2 時，非空層至多 r1+r2−1，另加一次空樹基例。

每層 O(1) 工作，故時間與遞迴堆疊皆為 $O(1 + \log(n_1+1) + \log(n_2+1))$，是**最壞界**。14 點數例的 rank 和為 4，恰做 3 次根比較；一般也可能提早結束。

刪最小：刪根後，左右子堆重新 meld

延續 14 點結果，移除根 2，合併根 5 與根 7。空堆先拒絕刪除；單點堆移除後回傳 null。

呼叫／回程	決策
meld(5,7)	保留 5，左子樹 8 不動，遞迴 meld(9,7)
meld(9,7)	保留 7，左子樹 11 不動，遞迴 meld(null,9)
meld(null,9)	回傳根 9，連同所有後代
回到 7	左 11、右 9 的 rank 皆 1，不交換，r(7)=2
回到 5	左 8、右 7 的 rank 皆 2，不交換，r(5)=3

結果為 `5:(8,7)`、`8:(10,50)`、`7:(11,9)`，其他邊保留。新右鏈 5→7→9，min=5，共 13 點。**刪除後 rank 反而增加並不矛盾**：rank 衡量形狀，並非隨元素個數單調變化。

插入與記號：別把不同的 rank 混在一起

insert 先建 `left=right=null, rank=1` 的單點，再 meld。向上一頁的 13 點堆插入新鍵 1：保留 1，meld(null,5) 回傳原堆；新根左 rank=0、右 rank=3，交換後原堆接在左側，新根 rank=1。結果 14 點，兩次呼叫、一次鍵比較，不需逐點搜尋原堆。

insert、delete-min 最壞皆為 $O(1+\log(n+1))$，find-min 看非空根為 O(1)。此處不計任意精度鍵比較與記憶體配置成本。

本章採 **null rank=0、leaf rank=1**。若另採 null npl=−1、leaf npl=0，對同一位置有 $npl = r - 1$；右鏈非空節點數為 npl+1，子樹至少有 $2^{npl+1} - 1$ 點。公式與基例須整套轉換。

這不同於 binomial／Fibonacci 的 degree（直接子節點數），或 splay 分析的 $\log_2 size$。它們衡量不同的量，不能一律加減 1 換算，也不能把右鏈界套成整樹高度界。

二項樹 B_k 的結構

- B_0 為單點。B_k 由兩棵 B_{k-1}，把其中一根連成另一根的子節點形成。
- 歸納得節點數 2^k、高度 k、根度數 k，根的子樹分別為 $B_0,\dots,B_{k-1}$。
- binomial heap 保留一組符合 heap order 的二項樹，每個 degree 最多一棵。
- n 的二進位表示決定有哪些 degree。例如 n=13=1101_2，包含 B_0,B_2,B_3。

Binomial meld 是二進位進位

- 先依 degree 合併兩個根清單，遇到相同 degree 的兩樹就比較根，把較大根連到較小根底下。
- 兩棵 B_k 形成 B_{k+1}，相當於同位元相加產生進位。遇到三棵同 degree 時保留一棵，再把另外兩棵 link。
- 根清單最多 $\lfloor \log_2 n \rfloor + 1$ 棵，標準 eager consolidation 的 meld 最壞 $O(\log n)$。
- 例如 $5 = 101_2$ 與 $3 = 011_2$ 相加，degree 0、1、2 連續進位，最後只剩 B_3，共 8 點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binomial 的 link 與專用插入

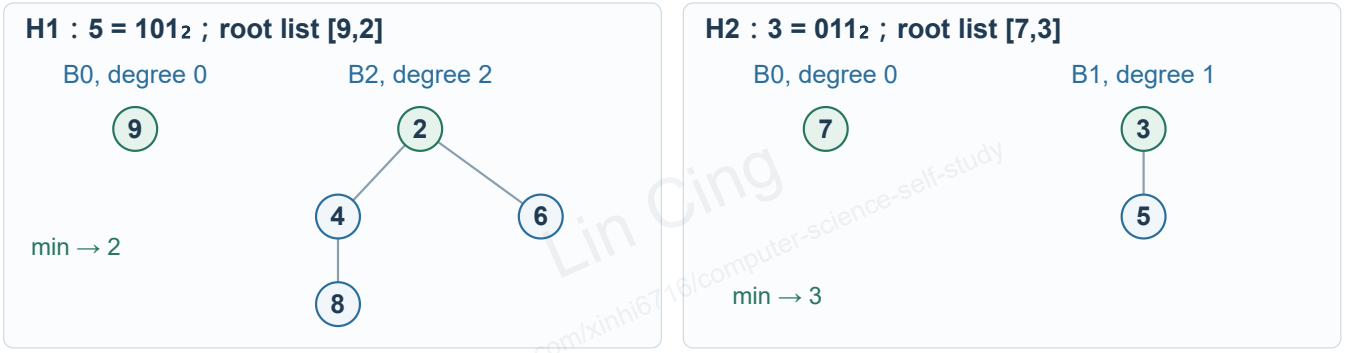
```
binomial_link(a,b):
    require degree(a)==degree(b)
    if a.key>b.key: swap(a,b)
    prepend b to a.child list; b.parent=a
    a.degree+=1; return a
binomial_insert(x):
    carry=singleton(x); d=0
    while tree[d] exists: carry=link(carry,tree[d]); clear tree[d]; d+=1
    tree[d]=carry; update min pointer
```

`swap(a,b)` 只交換局部根指標，不交換節點的鍵與 payload。link 的兩棵輸入都為 Bk，勝根原子 degree 為 k−1,⋯,0；把敗根 Bk 前插，便得到 k,k−1,⋯,0，正是 B(k+1) 的子樹組成。較小根留下，也維持新邊的 heap order。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binomial meld：輸入森林與串列順序

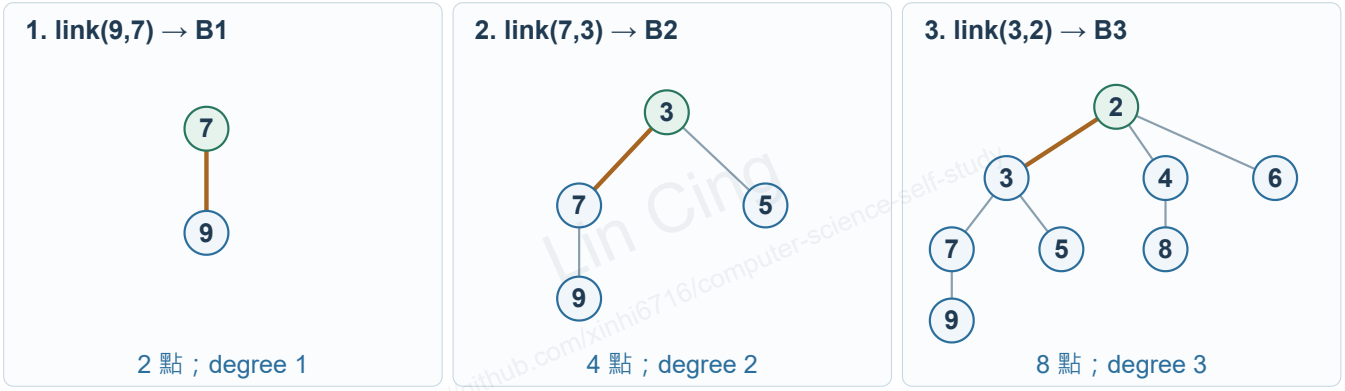
本例所有鍵互異。記法 `x:[子根…]` 按子串列順序列出，未列者為葉。H1 為 `9:[]`、`2:[4,6]`、`4:[8]`；H2 為 `7:[]`、`3:[5]`。根串列依 degree 遞增，子串列依 degree 遞減；link 把敗根連成勝根的**第一個子節點**。



H1 有 1+4=5 點，H2 有 1+2=3 點。根串列第一個根未必最小；H1 的最小鍵為 2，而非 9。

Binomial meld：三次比較與子樹合併

先 link 兩棵 B0 的根 9、7，再與根 3 的 B1 合併，最後與根 2 的 B2 合併。每次只比較兩個根；較大根原有的全部後代一併移動。



棕色邊是該次新增的 parent→child；其他連線保留。最後 2 的子根依序是 3、4、6，degree 分別為 2、1、0。八個鍵各出現一次，沒有搬換 payload，也沒有遺失敗根的子樹。

Binomial meld：每一個 degree 留下什麼？

步驟	本次 link	尚存根：key(degree)	根數
輸入	無	H1: 9(0),2(2)；H2: 7(0),3(1)	4
degree 0	9 與 7 → 7	7(1),3(1),2(2)	3
degree 1	7 與 3 → 3	3(2),2(2)	2
degree 2	3 與 2 → 2	2(3)	1

一次 link 少一棵樹、不減節點；根數從 4 降到 1，恰好做 3 次 link。它同時保持兩個不變量：相同 degree 的 B_k 合成 B_{k+1}，節點數為 $2^k + 2^k = 2^{k+1}$；較小根留下，所以所有新舊邊都符合 min-heap order。最後只剩 degree 3，也吻合 5+3=8=1000_2。

這是本例的操作次數，不能推出一般 meld 為 O(1)。一般演算法還要走訪來源根串列，合法 binomial heaps 的根數為 O(log n)；Fibonacci heap 的未整併根數可能是 Θ(n)，不能直接套用。

三棵同 degree，為什麼不能全部 link 成一棵？

以 7=111_2 加 3=011_2 為例；每欄最多有兩個來源根和一個低位進位。

degree	H1 的根數	H2 的根數	低位進位	此位保留	送往下一位
0	1	1	0	0	1
1	1	1	1	1	1
2	1	0	1	0	1
3	0	0	1	1	0

degree 1 的三棵 B1 共 6 點，必須保留一棵 B1，再把兩棵 link 成 B2；6 不是 2 的冪，不能把三棵當成一棵二項樹。結果保留 B1 與 B3，節點數 2+8=10，對應 1010_2。保留哪一棵可能改變最終樹形；合法結果不必唯一，但 degree、節點集合與 heap order 都必須正確。

Binomial 其餘操作的條件

- 專用 insert 把單點 B_0 加入並沿低 degree 進位，進位停止即結束，最壞 $O(\log n)$。以進位次數計帳，可得攤銷 $O(1)$；若每次都呼叫完整掃根清單的 meld，不能直接沿用此插入攤銷界。
- delete-min 先找最小根，移除該根，把其子樹清單按 degree 反轉後 meld，最壞 $O(\log n)$。
- find-min 若每次掃根，為 $O(\log n)$；額外維護最小根指標後可為 $O(1)$。
- decrease-key 可沿父鏈交換內容，最壞 $O(\log n)$。若 handle 指向節點身分，需同步處理內容交換對 handle 語義的影響。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binomial meld：以 degree 表處理進位

```
meld_roots(R1,R2):
    by_degree=empty
    for x in concatenation of detached root snapshots R1,R2:
        while by_degree[x.degree] contains y:
            clear by_degree[x.degree]
            x=binomial_link(x,y)
        by_degree[x.degree]=x
    return roots in degree order; recompute minimum pointer
```

每個 snapshot 中的根只作為輸入一次。碰撞時先清空舊 degree 格，再 link 並以新 degree 重試；每次少一個根，所以總 link 數不超過輸入根數減一。合法來源森林只有 O(log(n1+n2)) 個根，故走訪與進位共 O(log(n1+n2))。這個根數界不能套到未整併的 Fibonacci heap。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

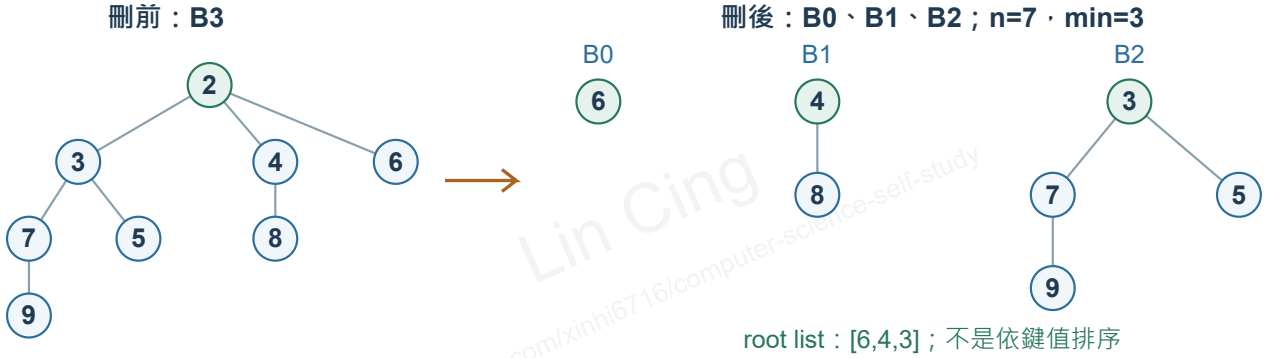
Binomial 刪根與降鍵：恢復 parent 和 min

```
delete_min():
    require roots not empty
    z=minimum root
    remove z from roots
    children=detach z.children; clear their parent pointers
    release z
    roots=meld_roots(roots,children)
decrease_key(x,new):
    require new<=x.key; x.key=new
    while x.parent!=null and x.key<x.parent.key:
        swap complete key-value contents and update handle mapping
        x=x.parent
    update minimum pointer
```

刪根先摘出子樹、清其 parent，再釋放舊根；meld 最後重算 min。降鍵可交換完整 key-value 內容，但若 handle 指向邏輯元素，handle→node 對應也要隨內容更新，不能讓下次操作找錯元素。

刪最小值：從同一棵 B3 拆回森林

刪除上一個合併結果的根 2，保留其三棵完整子樹。原子串列依 degree 為 2→1→0；改成根串列時反轉為 0→1→2，並把三個新根的 parent 設為 null。



新的根串列為 [6,4,3]，其樹分別有 1、2、4 點，總數 $7=111_2$。此例沒有其他根，degree 已互異，不需再 link；掃描三個根重設 min=3。反轉的理由是**degree 順序**，不是鍵值排序；degree-table 版可不反轉輸入，但輸出根串列仍依本章契約排列。

Fibonacci heap 延後整併

- Fibonacci heap 保留 heap-ordered forest、根循環雙向串列、最小根指標、每點 degree 及 mark。
- insert 只把單點加進根串列；meld 拼接根串列並更新最小根。這兩項不立即把相同 degree 整併。
- delete-min 將其子節點提升成根，再以 degree 表連結同 degree 根，最後重建最小根指標。
- 節點可失去部分子節點，所以其樹不是固定形狀的二項樹。degree 上界需要 cascading cuts 維持。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Decrease-key 與 cascading cuts

1. 降低 x.key。若仍不小於父鍵，不需切斷。
2. 若破壞 heap order，把 x 從父 p 切下，加到根串列，清除 x.mark。
3. 若 p 不是根，第一次失子把 p 標記；若它已被標記，再把 p 切下並向上處理。

非根節點在成為別人的子節點後，至多失去一個子節點而仍留在原父底下。根不需要因失子而作同樣標記。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci heap 的整併與標記操作

```
link_equal_degree_roots(a,b):
    require a.degree==b.degree
    if a.key>b.key: swap(a,b)
    roots=ring_remove(roots,b)
    a.child=ring_prepend(a.child,b)
    b.parent=a; b.mark=false; a.degree+=1
    return a
cut(x,p):
    p.child=ring_remove(p.child,x); p.degree-=1
    x.parent=null; x.mark=false; roots=ring_prepend(roots,x)
cascade(p):
    g=p.parent
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci heap 的整併與標記操作 （續）

```
    if g==null: return
    if not p.mark: p.mark=true
    else: cut(p,g); cascade(g)
decrease_key(x,new):
    require new<=x.key; x.key=new
    p=x.parent
    if p!=null and x.key<p.key: cut(x,p); cascade(p)
    if x.key<min.key: min=x
delete_min():
    require min!=null
    return extract_root(min)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

整併前若四個singleton根鍵為[7,3,8,5]，依此順序處理：7占degree0；3遇7，link成3→7、degree1；8占degree0；5遇8，成5→8、degree1，再遇3，成根3，子7及5→8，degree2。此表內最多一根每degree；必須以roots快照或安全迭代避免link時修改正在走訪的串列。

Fibonacci heap 的整併與標記操作 （續）

連鎖切斷用合法狀態中已標記的非根p、g示範：根r=2，其下g=3，其下p=5，其下x=7，g.mark=p.mark=true。降x到1，依序cut x、p、g，三者成根，r保留。t由1變4、m由2變0，Φ=t+2m由5降4，ΔΦ=−1；實際三次cut的固定成本以縮放勢能支付，端點常數留在攤銷O(1)。此例只展示標記記帳；degree大小的一般證明仍用後續連入順序與最多失一子的歸納，不能由四點圖取代。

insert只加未標記singleton根並更新min；meld拼接兩個根雙向環且來源失去所有權。空delete-min必拒絕，最後一節點被移除後min=null。任意節點刪除採後續的erase(handle)與extract_root完整程序，不修改鍵值、不指定哨兵最小鍵。即使目標大於父鍵，直接刪除造成的失子仍要觸發cascade，才能保留degree證明的前提。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙向環與共用整併的完整修復

```
ring_remove(head,x):
    if x.right==x: head=null
    else:
        x.left.right=x.right; x.right.left=x.left
        if head==x: head=x.right
    x.left=x.right=x; return head
ring_prepend(head,x):
    require x.left==x and x.right==x
    if head!=null:
        x.left=head.left; x.right=head
        head.left.right=x; head.left=x
    return x
snapshot_of_root_ring():
    S=[]
    if roots==null: return S
    x=roots
    repeat: S.append(x); x=x.right
    until x==roots
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙向環與共用整併的完整修復 （續）

```
    return S
consolidate():
    A=[]                        // missing degree entries mean null
    for x in snapshot_of_root_ring():
        if x.parent!=null: continue
        while len(A)<=x.degree: A.append(null)
        while A[x.degree]!=null:
            y=A[x.degree]; A[x.degree]=null
            x=link_equal_degree_roots(x,y)
            while len(A)<=x.degree: A.append(null)
        A[x.degree]=x
    roots=null; min=null
    for x in nonnull entries of A:
        x.left=x.right=x; x.parent=null; x.mark=false
        roots=ring_prepend(roots,x)
        if min==null or x.key<min.key: min=x
```

ring_remove的前提是x確實在head指定的環內；單點環回傳null，多點環接回左右鄰居，若摘掉代表點便改用其右鄰。摘下後x自成單點環，才能交給ring_prepend。這兩個操作只改固定數目指標，不搜尋前驅。它們同步用於根環與子環，因此cut、提升與link都有可重建的連結步驟。

雙向環與共用整併的完整修復 （續）

整併先保存根節點指標快照，避免link改環時破壞目前走訪。相同degree的根比較鍵，較大根自根環摘下、接到較小根子環、清mark並增加父degree；這就是前頁的link_equal_degree_roots。先清A舊格，再按新degree反覆進位，直到該格為空。A中每個非空格最後恰保留一棵合法堆序樹。

最後不再沿舊根環走訪，而依A逐根重建環及min；不能釋放z後讀舊min.key，也不能因z不是舊min便省略新根的比較。degree表按需補空格，總長O(1+log(n+1))；以t′表示cut/cascade後根數、d表示目標degree，本快照版另用O(t′+d)暫存指標，最壞O(n)，沒有宣稱額外空間只有degree表。單次實際工作同樣可能Θ(n)，攤銷時間與當下空間是不同指標。

Fibonacci degree 上界的理由

- 將 x 仍保留的 k 個子節點按連入時間排序。第 i 個連入時，x 至少已有 i−1 個子，因此該子當時 degree 至少 i−1。
- 它留作 x 的子時至多失去一子，所以目前 degree 至少 $\max(0,i-2)$。
- 用最小子樹大小遞迴可得 degree k 的節點至少有 F_{k+2} 點，故 $k=O(\log n)$。
- 這個下界連結「允許失子數」與「degree 不會太大」。省略 cascading cuts 就不能沿用此證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從子節點限制到 Fibonacci 下界

- 令 S(k) 為 degree 至少 k 節點的最小子樹點數，包含自身。這個定義使 S(k) 隨 k 不減，才能將子節點 degree 的下界代入。S(0)=1，S(1)≥2。
- degree 至少 k 的節點，取前 k 個保留子節點，按連入次序第 i 子的 degree 至少 $\max(0,i-2)$，所以對 $k\geq 2$ 有 $S(k)\geq 1+2S(0)+\Sigma_{j=1}^{k-2}S(j)$。
- 以歸納假設 $S(j)\geq F_{j+2}$ 代入，得到 $S(k)\geq 3+\Sigma_{j=1}^{k-2}F_{j+2}=F_{k+2}$，最後等式來自 Fibonacci 前綴和。
- 因 $F_{k+2}\geq \phi^k$、$\phi=(1+\sqrt{5})/2>1$，n 點堆中的任何 degree 皆滿足 $k\leq \log_\phi n$。這才是整併 degree 表只需 O(log n) 格的依據。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

勢能法把延後工作入帳

- 設 $\Phi=t+2m$，t 為根數，m 為被標記的非根節點數。攤銷成本為 $\hat{c}=c+\Phi_{after}-\Phi_{before}$。
- insert 增一個根，只增加常數勢能。meld 的勢能可加，因此維持常數攤銷成本。
- decrease-key 若切 c 個已標記祖先，每次切斷增一根但清一標記，勢能約下降 1，支付連續切斷的實際成本。
- 實作中每次 link/cut 的固定指標成本，可把 Φ 整體乘足夠常數支付。若從空堆開始且 $\Phi\geq 0$，總實際成本不超過總攤銷成本加上初始勢能。這是序列最壞保證，不是隨機平均。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci 的共同勢能與完整攤銷界

Fibonacci decrease-key若共切k≥1點，根數增加k。除了首切以外，每個繼續被切的祖先原來都已標記，至多最後停止祖先新增一個mark，所以 $\Delta m\leq 1-(k-1)=2-k$，$\Delta(t+2m)\leq 4-k$。若實際成本$\leq ak+b$，取縮放勢能 $A(t+2m)$、$A\geq a$，攤銷成本$\leq ak+b+A(4-k)\leq b+4A$，是與k無關的常數。完全不切時只比較／更新鍵與最小指標，也為常數。

刪最小根前t根、該根d子，degree上限D；實際成本$\leq a(t+d+D)+b$。提升子、link及重設根mark不增加標記數，整併後根數$\leq D+1$，故 $\Delta\Phi\leq A(D+1-t)$。取$A\geq a$後，攤銷成本$\leq a(d+D)+A(D+1)+b=O(D+1)=O(1+\log(n+1))$，使用$d\leq D$。這補足「勢能支付掃根」的代數，不把單次$\Theta(n)$工作說成最壞對數。

- Fibonacci heap 使 Dijkstra 的抽象成本達 $O(E+V\log V)$，前提為正確使用 handle 與 decrease-key，且圖權重符合 Dijkstra 條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不修改鍵值的 erase 與任意根移除

```
erase(h):
    z=resolve_live_owned_handle(h)   // reject stale/foreign handle first
    if z.parent!=null:
        p=z.parent; cut(z,p); cascade(p)
    return extract_root(z)
extract_root(z):
    require z is a live root owned by this heap
    result=copy(z.item)             // fixed-size nonthrowing model
    while z.child!=null:
        c=z.child
        z.child=ring_remove(z.child,c); z.degree-=1
        c.parent=null; c.mark=false; roots=ring_prepend(roots,c)
    roots=ring_remove(roots,z); min=null
    invalidate_handle(z); release(z); n-=1
    if n==0: roots=null; return result
    consolidate()
    return result
```


不修改鍵值的 erase 與任意根移除 （續）

每個節點保存key、item、parent、child、degree、mark與雙向環的left/right；roots、child分別是根環與子環的代表指標，空環為null。handle指向特定活節點，不是「目前等於某鍵的任一節點」。erase只使目標handle失效；即使兩節點鍵相同，也不搬換其餘節點的item或身分。外部不透明handle必先經所屬容器的活節點登錄表驗證，再取得節點；不能先解參照已釋放raw pointer檢查alive。本節成本以已持有合法handle為前提，額外的查找／驗證成本另計。

非根目標z先切成根，並把原父p失去一子的事件交給既有cascade：p非根且未標記則標記；已標記則切p並繼續到祖父；到根即停止。這次cut不是因鍵變小，而是因即將刪除節點；省略cascade會允許非根連失兩子，破壞Fibonacci degree下界。原本已是根的z直接進extract_root，不因刪掉根而任意標記其子。

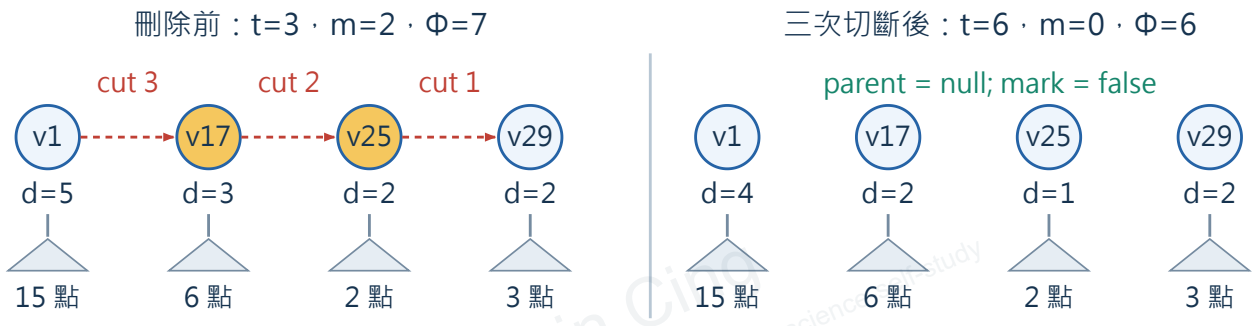
extract_root不要求z=min。逐一先從z子環摘下c、減degree、清parent/mark，再接根環；整棵c子樹保留。移除z後先清min以避免保留懸空最小指標，再使其handle失效及釋放，僅減n一次。仍有節點就整併全部根並重算真正min，不把目標假裝成最小鍵。空堆delete_min先拒絕；空堆沒有合法erase handle。刪最後一點時子環必空，n=0、roots=min=null；再次使用舊handle須拒絕。meld後可接受哪些handle由容器的所有權契約決定，不能繼續對失去所有權的來源堆刪除。

Erase 的合法起始狀態與三級切斷

```
for i=0..32: h[i]=insert(i)     // v_i is permanent identity, initially key i
delete_min()                    // removes v_0; consolidates to root v_1
decrease_key(h[18],-10)
decrease_key(h[26],-11)
erase(h[29])                    // key stays 29 until release
```

根與子環都以前插建立，快照沿right走訪。前33次插入後刪v0，得到degree5的根v1。其子v17原有子v25,v21,v19,v18；v25原有子v29,v27,v26；v29有v31,v30，v31有v32。將v18降至−10使它切離並標記v17；將v26降至−11使它切離並標記v25。刪除前根為v26(key−11,d0)、v18(key−10,d0)、v1(key1,d5)，標記者v17、v25。目標v29不是最小值，且仍有兩棵子樹。

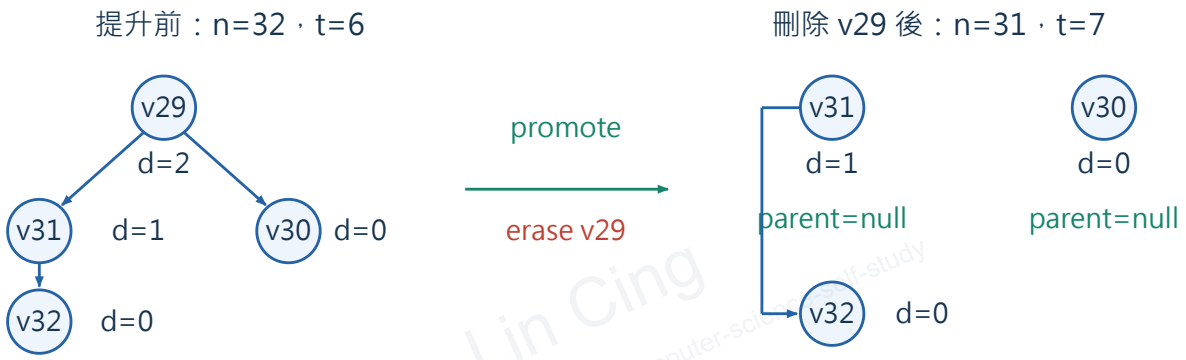
Erase 的合法起始狀態與三級切斷 （續）



另有根 v18(key −10)、v26(key −11)；金色表示 mark=true。
三角形代表保留的子樹群；點數不含上方節點。兩態皆有 n=32。

四個顯示節點、15／6／2／3 點的子樹群及另兩個根合計 32 點。切斷只改 parent 連結，保留所有節點身分；已標記的祖先切成根後清除 mark，到根 v1 時停止，不把 v1 標記。

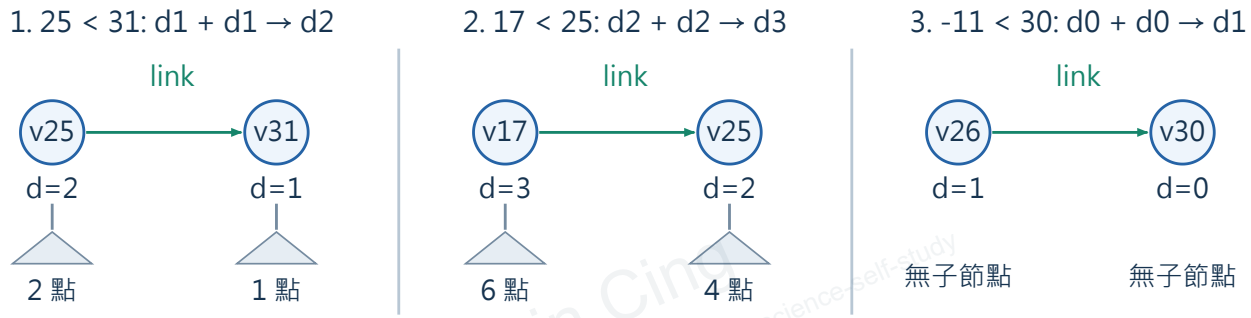
提升目標的子樹與移除唯一節點



其餘五根 v1、v17、v25、v26、v18 共 28 點不變；只有 h29 失效。

移動 v31 時保留其子 v32，因此它仍是兩點樹；加上單點 v30，提升的兩棵樹共有三點。唯一移除的是 v29，key=−11 的 v26 仍是最小節點，其餘 handle 仍指向原節點。

依 degree 進位的三次 link



A[0]=v18, A[1]=v26, A[3]=v17, A[4]=v1; min=v26 (key −11)
最終四棵樹的點數：1 + 2 + 12 + 16 = 31；t=4，m=0，Φ=4。

第一個 link 使 v25 成為 degree=2 的五點樹；再接到 v17 下方，得到 degree=3 的十二點樹。v30 接到 v26 下方後形成 degree=1 的兩點樹。未變動的根 v18、v1 分別保留一點與十六點。

Erase 的狀態表與身分守恆

已完成步驟	根數t	標記非根數m	Φ=t+2m	必要狀態變化
刪除前	3	2	7	v29.parent=v25，v25.parent=v17
cut v29	4	2	8	v25.degree由2降1
cascade切v25	5	1	7	v25清mark，v17.degree由3降2
cascade切v17	6	0	6	v17清mark，v1.degree由5降4
提升v31、v30	8	0	8	兩者parent=null，v29.degree=0
摘除並釋放v29	7	0	7	n由32降31，h29失效
三次link後	4	0	4	degree各異，重選min=v26

整併快照依序是v30(d0),v31(d1),v17(d2),v25(d1),v26(d0),v18(d0),v1(d4)。v25與v31同degree，v25贏得v31而升d2；再遇v17，v17贏v25而升d3；v26(key−11)贏v30而升d1。degree表最後0→v18、1→v26、3→v17、4→v1，共31節點、min仍為key−11。v31與v30只在中間狀態被提升為根，之後又合法成為子，handle仍指原節點；不能把「曾提升」誤解成最終一定是根。

任意刪除的正確性與攤銷推導

```
// Potential Phi_A=A*(t+2*m), with a constant A large enough for pointer work.
// Cut/cascade phase: k cuts, actual <= a*k+b, Delta(t+2m)<=4-k.
// Root extraction: degree d, initial t' roots, final at most D(n)+1 roots.
// Actual <= a*(t'+d+D(n)+1)+b; marks never increase in this phase.
```

正確性分三段。第一，切出z只刪一條父子邊，原有子樹內堆序不變；cascade嚴格執行「非根首次失子標記、再次失子即切斷」，維持degree證明所需的歷史限制。第二，提升z每個子樹保留其完整元素集合，唯一移除的資料就是z.item；移除前保存結果，移除後只有z的handle失效。第三，link只以較小根為父、等degree進位，不丟節點且保留堆序；從最後所有根取min即全域最小。即使z不是舊min，三段論證都不改變。

切斷階段若有k≥1次cut，根數增加k；除了首次切z之外，被繼續切的祖先原先都已標記，最後至多新標記一個祖先，故Δm≤2−k、Δ(t+2m)≤4−k。實際成本≤ak+b，取縮放係數A≥a，攤銷≤ak+b+A(4−k)≤b+4A=O(1)。z原本是根則k=0，這一段只做常數檢查。

任意刪除的正確性與攤銷推導 （續）

令切斷後有t′根，被刪根degree為d，整併所需最大degree上界為D(n)=O(log(n+1))。extract_root實際成本≤a(t′+d+D(n)+1)+b；提升與link都只清mark，最後根數≤D(n)+1，所以該段ΔΦ_A≤A(D(n)+1−t′)。加總並用A≥a及d≤D(n)，得到攤銷≤a(d+D(n)+1)+A(D(n)+1)+b=O(1+log(n+1))。兩段勢能差相接望遠鏡消去，整個erase即O(1+log(n+1))攤銷；不是對每段分別挑無法相容的勢能。

此操作不保證單次對數：長級聯、大量根快照／掃描都可花Θ(n)。從空堆的合法操作序列以同一非負Φ_A計帳；任意初始狀態另保留Φ初值。結果／鍵比較與節點釋放按固定成本模型，合法handle的查找成本不隱藏在其中。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pairing heap 的簡單 link

- 標準 pairing heap 是多子 min-heap。meld 比較兩根，把較大根作為較小根的新子節點，只改固定數目指標。
- insert 是與單點 meld。decrease-key 通常在降低後把該節點與其子樹切下，再與根 meld。
- 找到並從兄弟串列切下需要額外 sibling/parent 或前驅資訊。只有單向 next 而沒有定位資訊，切斷不一定常數時間。
- 操作前須確認 handle 存活且屬於此堆；固定指標步數不代表 decrease-key 有 Fibonacci heap 的 `O(1)` 攤銷界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pairing 的 sibling 反向連結與切斷

```
// Pairing roots passed to link have parent=prev=next=null.
detach(x):
    p=x.parent
    if x.prev==null: p.first_child=x.next
    else: x.prev.next=x.next
    if x.next!=null: x.next.prev=x.prev
    x.parent=x.prev=x.next=null
link(a,b):
    if a==null: return b
    if b==null: return a
    if a.key>b.key: swap(a,b)
    b.parent=a; b.prev=null; b.next=a.first_child
    if b.next!=null: b.next.prev=b
    a.first_child=b; return a
```

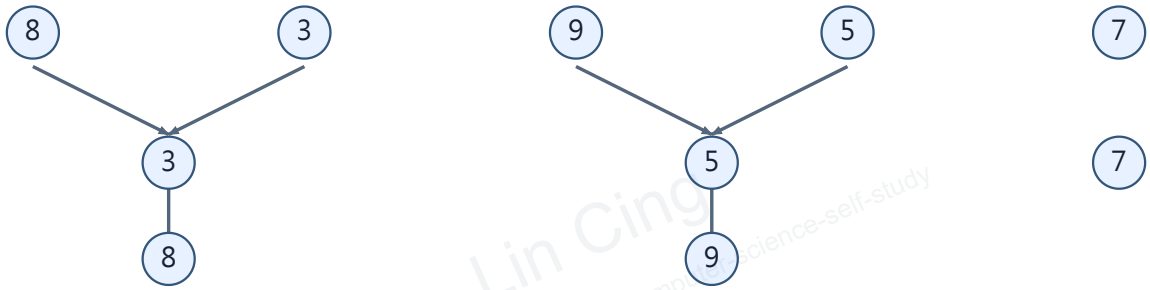
每個節點明存 parent、前兄弟 prev、後兄弟 next 與 first_child。detach 只用於非根x；first_child 的 prev=null，其餘兄弟的 prev 指前一兄弟。解除第一子必改父的first_child，解除其他子必改前兄弟的 next，兩者都要改後兄弟的prev。x的整個子樹原封保留，故降低鍵後可與根link。根本身降鍵不需 detach。刪根的第一輪配對前，逐子保存next並清parent/prev/next，才能把各子當作獨立根交給link。

Pairing 的完整兩輪程序與延後成本

```
link(a,b):
    if a==null: return b
    if b==null: return a
    if a.key>b.key: swap(a,b)
    prepend b to a.children; return a
delete_min():
    require root!=null
    children=detach_all_children(root); release(root)
    paired=[]
    for adjacent (a,b) left-to-right: paired.append(link(a,b))
    if odd last child remains: paired.append(last)
    root=null
    for heap in reverse(paired): root=link(heap,root)
decrease_key(x,new):
    require new<=x.key; x.key=new
    if x!=root: detach x in O(1) using backlinks; root=link(root,x)
```

Pairing 的完整兩輪程序與延後成本 （續）

Pairing 第一輪：相鄰兩兩比較，保留未配對7



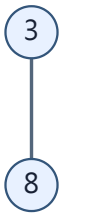
子根8,3,9,5,7先配(8,3)得3含子8、(9,5)得5含子9，7獨留。右至左先link(7,空)=7，再link(5,7)得5含子7,9，最後link(3,5)得3含子5,8。故最終3的子5仍保留7,9，沒有把被配對子樹拆散。每次delete-min的當下實際link數與舊根degree同階，可為Θ(n)。

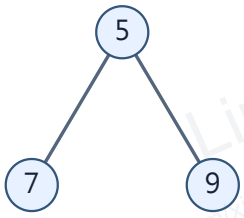
Pairing 第二輪保留子樹的實際形狀

```
paired=[root3(child8),root5(child9),root7]
acc=link(root7,null); acc=link(root5,acc)
acc=link(root3,acc)       // 兩輪共 4 次非空 link
```

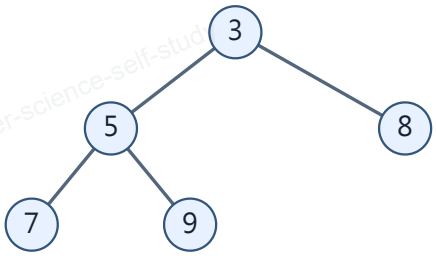
第二輪由右至左，整棵子樹一起合併

link(5,7) → 5



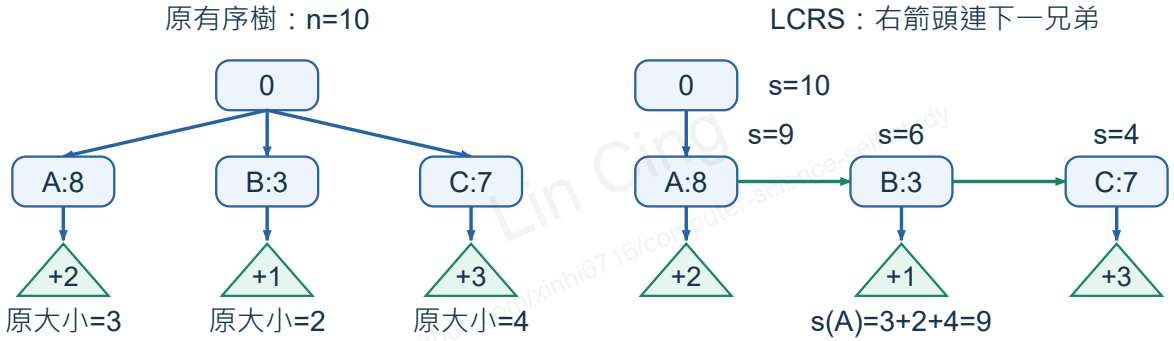


link(3,5) → 3



LCRS 表示與非負勢能

把有序多叉樹改看成 left-child/right-sibling（LCRS，左子／右兄弟）二元樹。令 s(v) 是從 v 沿這兩種指標可達的節點數，ρ(v)=log₂s(v)，Φ=Σ_vρ(v)。s(v) 包含右側兄弟的子樹，**不是原多叉樹中 v 的子樹大小**。所有 s(v)≥1，所以 Φ≥0；空堆 Φ=0。這些量僅供分析，不在操作時重新計算。



△ +j：省略的後代共 j 點，不含上方根

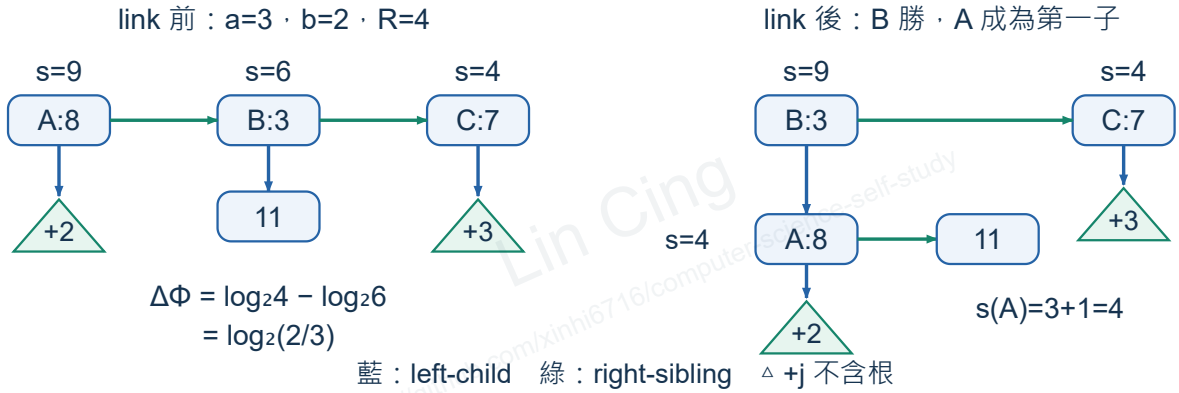
例中原根0有三棵子樹，大小3、2、4；二元表示中 s(A)=9、s(B)=6、s(C)=4，根的 s=10。刪根後，子樹按原順序形成 forest（森林），只有根的 log₂10 被移除，其餘 s 不變。

LCRS 表示與非負勢能 （續）

數個獨立堆的總勢能是各堆勢能之和，根之間不接 right-sibling。delete-min 內部則按剩餘分量順序定義暫時 forest 的虛擬兄弟鏈。即使用陣列暫存已清 next 的根，也用分量順序解讀此虛擬鏈；不把逐次清指標當作另一種堆勢能。每次宏觀 link 包含常數次解除／重接，掃描、暫存及最後釋放根共 O(k+1)，兩端勢能均與實際堆一致。

相鄰 link 只改兩個根的秩

考慮同一 forest 中相鄰的 A、B，其原多叉子樹大小為 a、b≥1，右方其餘分量共有 R≥0 點。link 前 s(A)=a+b+R，s(B)=b+R。較大鍵的根成為勝根的**第一個子節點**；相等鍵固定採任一一致規則。



link 後勝根的 s=a+b+R，敗根的 s=a+b−1。其他節點的 s 不變：左側根仍能到達同一批節點，舊子鏈內部也未改變。因此無論誰贏，

$\Delta\Phi_link = \log_2(a+b-1) - \log_2(b+R)$。

第一輪的正後綴不等式

令 A=a+b、T=A+R。當 R>0，由相鄰 link 公式得到

$1+\Delta\Phi_link = \log_2[2(A-1)/(b+R)] \le \log_2(2A/R) \le 2\log_2(T/R)$。

第一個不等式用 $A-1 \le A$、$b+R \ge R$；第二個等價於 $2AR \le (A+R)^2$，因右減左為 $A^2+R^2 \ge 0$。此推導只用大小，不依賴根鍵比較的勝負。

第一輪由左至右處理第 j 對，記其剩餘總量 $T_j = a_j + b_j + R_j$。尚未處理的右側完整保留，故相鄰兩對滿足 $R_j = T_{(j+1)}$。把各次不等式相加時，$\log T_{(j+1)}$ 與 $-\log R_j$ 抵消，留下首尾項。R=0 的最後一對不能代入除以 R 的式子，下一頁分開處理。

奇數個子根的第一輪望遠鏡和

刪除前 n 點，刪根後 N=n−1 點。原根有 k=2q+1 個子節點，第一輪做 q 次 link，最右一棵大小 r≥1 的樹不配對。每一對右方都至少有這棵樹，因此每次 R_j>0。

$q+\Delta\Phi_1 \le 2\Sigma_{(j=1)}^{q} \log_2(T_j/R_j) = 2\log_2(N/r) \le 2\log_2 N$。

所以 $\Delta\Phi_1 \le -q+2\log_2 N$。負的 −q 項將用來支付線性數目的 link；只寫 O(log n) 的勢能增加會漏掉這項必要抵銷。

既有五子例8,3,9,5,7皆為單點：N=5、q=2、r=1。兩次 link 的精確勢能差為 $\log_2(1/4)$、$\log_2(1/2)$，合計 −3。若 k=1，q=0、r=N，兩邊空和均為0，第一輪沒有 link；不需引入 log0。

偶數個子根的最後一對

k=2q≥2 時，最右一對沒有右後綴，R_q=0。令該對總大小 A_q=a_q+b_q≥2。前 q−1 對仍可望遠鏡相加：

$(q-1)+\Delta\Phi_{前} \le 2\log_2(N/A_q)$。

最後一對直接使用精確公式：$\Delta\Phi_{末}=\log_2(A_q-1)-\log_2 b_q \le \log_2 A_q$。相加得到

$\Delta\Phi_1 \le -q+1+2\log_2(N/A_q)+\log_2 A_q \le -q+1+2\log_2 N$。

q=1 時前段沒有 link，N=A_q，空和上界為0，公式仍成立。奇偶兩種情況可統一為 $\Delta\Phi_1 \le -q+2\log_2 N+1$，其中 $q=\lfloor k/2 \rfloor$；k=0 另作單點基例。

第二輪與完整 delete-min 攤銷界

第一輪留下 $m=\lceil k/2 \rceil$ 棵樹。第二輪由右至左，令左邊待接樹大小 a、已累積右樹大小 b，故 $R=0$。每次 $\Delta\Phi_link=\log_2(a+b-1)-\log_2 b \le \log_2((a+b)/b)$。

下一次的累積量就是本次 $a+b$；因此 $\Delta\Phi_2 \le \log_2(N/b_0) \le \log_2 N$，其中 $b_0 \ge 1$ 是最右分量大小，$m \le 1$ 時沒有 link。五子例的兩次精確比值為2、4/3；連同第一輪及移除原根的 1/6，整次勢能比值為 $(1/6)(1/4)(1/2)(2)(4/3)=1/18$。

$k \ge 2$ 時，移根再加兩輪：$\Delta\Phi \le -\log_2 n - q + 3\log_2 N + 1 \le -q + 3\log_2 n + 1$。實際工作 $\le ck+d$，且 $k \le 2q+1$。固定選同一縮放勢能 $\Psi=C\Phi$、$C \ge 2c$，則實際+$\Delta\Psi \le (2c-C)q + 3C\log_2 n + c + d + C = O(1+\log n)$。這同時支付掃描及兩輪共 $k-1$ 次 link，並非只支付第一輪。

$k=0$ 時 $n=1$，刪後空堆，$\Delta\Phi=0$、實際 $O(1)$；$k=1$ 時只把唯一子樹當根，$\Delta\Phi=-\log_2 n$、實際 $O(1)$。空堆 delete-min 拒絕。陣列式程序暫存 $O(k)$ 根，可能 $O(n)$ 空間；此證明不宣稱 $O(\log n)$ 暫存空間。

同一勢能下的 meld、insert 與 decrease

meld 兩個獨立非空堆，大小 a、b，$n=a+b$。link 前兩根各為獨立根，秩為 $\log_2 a$、$\log_2 b$；之後勝根 $s=n$、敗根 $s=n-1$，其餘不變。故

$\Delta\Phi_meld=\log_2[n(n-1)/(ab)] \le \log_2 n$，因 $ab \ge a+b-1=n-1$。任一堆為空則 $\Delta\Phi=0$。insert 是與單點 meld：非空插入的 $\Delta\Phi=\log_2 n$，空堆第一點 $\Delta\Phi=0$。雖兩操作當下均 $O(1)$，**此勢能分析對它們收 $O(1+\log(n+1))$ 攤銷**。

decrease-key 若目標已為根，改鍵不改形狀、$\Delta\Phi=0$。非根 x 的原多叉子樹大小為 a。先切為兩個獨立堆：x 的 s 失去右兄弟後綴，原來能到達該子樹的二元祖先也只會減少可達點，其餘 s 不變，所以切斷的 $\Delta\Phi \le 0$。再 meld 回主堆，勢能增加 $\le \log_2 n$；使用正文 parent/prev/next 的合法 handle 才有 $O(1)$ 實際 cut。故同一 Ψ 下 decrease-key 收 $O(1+\log(n+1))$，不在此宣稱更強的 $O(1)$ 攤銷。

序列總成本與分析的適用範圍

操作	當下實際成本	本節同一 Ψ 的攤銷收費
find-min	O(1)	O(1)
insert	O(1)	O(1+log(n+1))
meld	O(1)	O(1+log(n+1))，n 為合併大小
decrease-key	O(1)，須合法 handle/backlink	O(1+log(n+1))
delete-min	O(k+1)，最壞 O(n)	O(1+log(n+1))

從空堆集合開始，$\Psi_0=0$、最終 $\Psi \geq 0$，Σ實際=Σ攤銷+$\Psi_0-\Psi_末 \leq \Sigma$攤銷。令 N 是最多存活點數，I、M、D、K、F 分別為插入、合併、刪根、降鍵、查最小的次數，得到 $O((I+M+D+K)(1+\log_2(N+1))+F)$ 總成本；另建空堆每次 O(1)。任意非空初態另加 Ψ_0。

本節完整證明的是標準 two-pass 的上述一組共同界。不能把其他更精細分析的 O(1) 攤銷 insert/meld 直接移進此總式，改稱 O(I+M+(D+K)log N+F)；也不能從 O(1) 當下 cut 推成 O(1) 攤銷 decrease-key。操作方向與 prepend-child 規則若改變，必須重新核對不等式。

複雜度比較與選型

結構	find-min	meld	delete-min	decrease-key
Leftist	O(1) 最壞	O(log n) 最壞	O(log n) 最壞	本章不提供快界
Eager binomial	O(1)，需 min 指標	O(log n) 最壞	O(log n) 最壞	O(log n) 最壞
Fibonacci	O(1) 最壞	O(1) 最壞	O(log n) 攤銷	O(1) 攤銷
Two-pass pairing	O(1) 最壞	O(1) 指標工作	O(log n) 攤銷	本節同一勢能 O(log n) 攤銷

表中 decrease-key 均以已持有合法 handle 為前提。不同操作的攤銷界須屬於同一套分析，不能各取最樂觀結論任意拼接。

所有對數成本在空／單點情況都保留常數項：例如兩空堆meld也要判斷指標，為O(1)；leftist精確上界寫O(1+log(n1+1)+log(n2+1))。delete-min只對非空堆定義；空輸入應回報錯誤，不套用log0。

參考文獻與進階閱讀

- Fibonacci heap：Fredman–Tarjan 論文。
- Pairing heap：原始論文與作者說明、Pettie 的進階分析。
- Binomial heap：Stanford CS166 課程。
- 標準 two-pass 變體的 prepend-child、配對方向與 backlink：Sinnamon–Tarjan, § 2。本章採用的 LCRS 勢能與各操作共同收費，已在前文完整推導。
- Chicago CS223：二項樹表示、first-child link 與根串列順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study